Notes of Reinforcement Learning[1]

Yue Wu

September 18, 2026

[1] Based on ORCS6529 at Columbia University, taught by Prof. Shipra Agrawal

Outline

- Introduction
  - MDP
  - Solve an MDP when MDP is known
    - L-operator
    - Value based Iteration
    - Q-value based iteration
    - Discussion
- RL: Solve an MDP with samples when MDP is unknown
  - Reinforcement Learning
  - Q-value based algorithm(and argmaxed-Q policy)
    - Q-learning
    - Discussion of Q-learning
    - Policy iteration with TD-learning
  - Policy based algorithm
    - Vanilla Policy gradient methods
    - Conservative Greedy policy improvement algorithm
    - Trust Region Policy Optimization
    - Proximal Policy Optimization

Outline

Introduction
- MDP
- Solve an MDP when MDP is known
  - L-operator
  - Value based Iteration
  - Q-value based iteration
  - Discussion

RL: Solve an MDP with samples when MDP is unknown
- Reinforcement Learning
- Q-value based algorithm(and argmaxed-Q policy)
  - Q-learning
  - Discussion of Q-learning
  - Policy iteration with TD-learning
- Policy based algorithm
  - Vanilla Policy gradient methods
  - Conservative Greedy policy improvement algorithm
  - Trust Region Policy Optimization
  - Proximal Policy Optimization

Markov Decision Process

$$(\mathcal{S}, \mathcal{A}, R, P)$$

- $\mathcal{S}$: State space
- $\mathcal{A}$: Action space
- R: Reward $S \times A \to \mathbb{R}$
- P: Transition prob $S \times A \times S \to [0, 1]$

If we have finite S states and A actions, then the dimension of Reward[2] is $R \in \mathbb{R}^{S \times A}$, the dimension of Transition prob matrix is $P \in \mathbb{R}^{S \times A \times S}$.

At $t = 0, 1, 2, ...$, observe s_t, take action a_t, observe reward r_t and next state s_{t+1}.

Markov property

$$\mathbb{P}(s_{t+1} = s' \mid s_t = s, a_t = a, \text{history}) = \mathbb{P}(s_{t+1} = s' \mid s_t = s, a_t = a) =: P(s, a, s')$$

$$\mathbb{E}(r_t \mid s_t = s, a_t = a, \text{history}) = \mathbb{E}(r_t \mid s_t = s, a_t = a) =: R(s, a)$$

[2] R(s,a)=$\sum_{s'} R(s, a, s')P(s, a, s')$

Formulating an MDP

For impossible action, use -inf reward or mask.

About policy

Policy: a mapping from state space to action space $\pi : \mathcal{S} \to \mathcal{A}$

- Markov property and Markovian[?] Due to Markovian property of the MDP, it suffices to consider only Markovian policies.
  If an optimal policy exists, there always exists a Markovian policy that is optimal.
- Dimension of a policy

	Deterministic	**Stochastic**
Non-stationary	$H \times S$	$H \times S \times A$
Stationary	$1 \times S$	$1 \times S \times A$

- Horizon and policy type
  - Finite horizon: Non-stationary policy
  - Infinite horizon(with average/discounted reward): such a MDP always has a stationary optimal policy, whenever optimal policy exists[?]

Goal of MDP - Maximize the expectation of total reward[4]

(1) Finite horizon (non-stationary policy)
Find a policy $\pi = (\pi_1, ..., \pi_H), \pi_t \in \Pi$ that maximize the expectation of the total reward of $t = 1, 2, ..., H$.

$$V^{\pi}(s) = \mathbb{E}\left(\sum_{t=1}^{H} \gamma^{t-1} r_t \mid s_1 = s\right)$$

$$\pi^* \in \arg\max_{\pi \in \Pi^H} V^{\pi}(s), \quad \forall s \in \mathcal{S}$$

(2) Infinite horizon (stationary policy) [3]
Find a stationary policy $\pi \in \Pi$ that maximize the expectation of the total reward of $t = 1, 2, ...$

$$V^{\pi}(s) = \mathbb{E}\left(\sum_{t=1}^{\infty} \gamma^{t-1} r_t \,\middle|\, s_1 = s\right), \ \gamma \in (0, 1)$$

$$\pi^* \in \arg\max_{\pi \in \Pi} V^{\pi}(s), \quad \forall s \in \mathcal{S}$$

[3] Unless stated otherwise, this setting is assumed throughout.

[4] The expectation is taken over trajectories induced by the environment transition dynamics and the policy $\mathbb{E}_{a \sim \pi(s),\, s' \sim P(s,a)}$.

Bellman equations

$$
\begin{aligned}
V^{\pi}(s) &= \mathbb{E}_{a\sim\pi(s),\, s'\sim P(s,a)}\left[R(s,a,s') + \gamma V^{\pi}(s')\right] \\
&= \sum_{a} \pi(s,a) \sum_{s'} P(s,a,s') \left(R(s,a,s') + \gamma V^{\pi}(s')\right) \\
&= \sum_{a} \pi(s,a) \Big(R(s,a) + \gamma \sum_{s'} P(s,a,s') V^{\pi}(s')\Big)
\end{aligned}
$$

Optimal Value and Bellman optimality equations

The **optimal value** is defined as

$$V^*(s) = \sup_{\pi} \mathbb{E}\left[\sum_{t=1}^{\infty} \gamma^{t-1} r_t \mid s_1 = s\right]$$

which satisfies the following **Bellman optimality equations**

$$V^*(s) = \max_a \left[R(s,a) + \gamma \sum_{s'} P(s,a,s') V^*(s')\right]$$

$$= \max_a \left[R(s,a) + \gamma \mathbb{E}_{s' \sim P(s,a)} \left[V^*(s')\right]\right]$$

- equations that connect $V^*(s)$ for certain s with the values starting from all states $V^*(s)$, $s \in \mathcal{S}$
- self-related(recursive) equations $\rightarrow$ can be solved via iterative algorithms

The **optimal policy** is therefore defined as:

$$\pi^*(s) := \arg\max_a \left[R(s,a) + \gamma \mathbb{E}_{s' \sim P(s,a)} \left[V^*(s')\right]\right]$$

- a policy that attains $V^*(s)$ for all s

How to find the optimal policy/value

When we know the (R, P) of MDP, this is just a simple set of equations to solve. But technically, how?

- Linear programming
- Iterative algorithms
  - Value iteration: Iteratively improve estimate of optimal value(a vector of length S) $[V^*(s),\ s \in \mathcal{S}]$
  - Policy iteration using values: Iteratively improve estimate of optimal policy(a vector of length S) $[\pi^*(s),\ s \in \mathcal{S}]$ by using values $[V^*(s),\ s \in \mathcal{S}]$)
  - Q-value iteration: Iteratively improve estimate of Q-values(a tabular of size $S \times A$) $[Q^*(s, a),\ s \in \mathcal{S},\ a \in \mathcal{A}]$
  - Policy iteration using Q-values: Iteratively improve estimate of argmaxed optimal policy $[\pi^*(s),\ s \in \mathcal{S}]$ by using Q-values $[Q^*(s, a),\ s \in \mathcal{S},\ a \in \mathcal{A}]$

Outline

Introduction
- MDP
- Solve an MDP when MDP is known
  - L-operator
  - Value based Iteration
  - Q-value based iteration
  - Discussion

RL: Solve an MDP with samples when MDP is unknown
- Reinforcement Learning
- Q-value based algorithm(and argmaxed-Q policy)
  - Q-learning
  - Discussion of Q-learning
  - Policy iteration with TD-learning
- Policy based algorithm
  - Vanilla Policy gradient methods
  - Conservative Greedy policy improvement algorithm
  - Trust Region Policy Optimization
  - Proximal Policy Optimization

Outline

Introduction
- MDP
- Solve an MDP when MDP is known
  - L-operator
  - Value based Iteration
  - Q-value based iteration
  - Discussion

RL: Solve an MDP with samples when MDP is unknown
- Reinforcement Learning
- Q-value based algorithm(and argmaxed-Q policy)
  - Q-learning
  - Discussion of Q-learning
  - Policy iteration with TD-learning
- Policy based algorithm
  - Vanilla Policy gradient methods
  - Conservative Greedy policy improvement algorithm
  - Trust Region Policy Optimization
  - Proximal Policy Optimization

L-operator and its contraction property

Define $L, L^\pi : \mathbb{R}^S \to \mathbb{R}^S$ as

$$[LV](s) := \max_a \left[R(s,a) + \gamma \sum_{s'} P(s,a,s') V(s') \right]$$

$$[L^\pi V](s) := \mathbb{E}_{a \sim \pi(s)} \left[R(s,a) + \gamma \sum_{s'} P(s,a,s') V(s') \right]$$

Contraction property of L-operator:

$$\|Lv - Lu\|_\infty \leq \gamma \|v - u\|_\infty.$$

$$\|L^\pi v - L^\pi u\|_\infty \leq \gamma \|v - u\|_\infty.$$

The introduction of L-operator makes the form of value/policy iteration more clear. And actually, the Convergence of value/policy iteration is based on the fact that L-operator is a Contraction Mapping.

Outline

Introduction
- MDP
- Solve an MDP when MDP is known
  - L-operator
  - Value based Iteration
  - Q-value based iteration
  - Discussion

RL: Solve an MDP with samples when MDP is unknown
- Reinforcement Learning
- Q-value based algorithm(and argmaxed-Q policy)
  - Q-learning
  - Discussion of Q-learning
  - Policy iteration with TD-learning
- Policy based algorithm
  - Vanilla Policy gradient methods
  - Conservative Greedy policy improvement algorithm
  - Trust Region Policy Optimization
  - Proximal Policy Optimization

Value Iteration

1. Initialize $V^0 \in \mathbb{R}^S$, specify $\epsilon > 0$
2. Repeat for k=1,2,... until $\|V^k - V^{k-1}\|_\infty \leq \epsilon \frac{1-\gamma}{2\gamma}$

$$V^k(s) = \max_a \left[R(s,a) + \gamma \sum_{s'} P(s,a,s') V^{k-1}(s') \right], \ \forall s$$

i.e.

$$V^k = LV^{k-1}$$

3. Output optimal value and optimal policy[5]

$$\pi^k(s) := \arg\max_a \left[R(s,a) + \gamma \sum_{s'} P(s,a,s') V^k(s') \right]$$

Convergence of value iteration:

$$\|V^k - V^*\|_\infty \leq \frac{\gamma^k}{1-\gamma} \|V^1 - V^0\|_\infty$$

$$\|V^{\pi^k} - V^*\|_\infty \leq \frac{2\gamma^k}{1-\gamma} \|V^1 - V^0\|_\infty$$

[5] different policies can have the same maximum value at some state, but we use '=' to indicate the selection rule when a state has the maximum

Policy Iteration using values

1. Initialize policy π^0
2. Repeat for k=1,2,... until $\pi^{k+1} = \pi^k$ (or not much change in policy)
   - (Policy evaluation) Compute $V^{\pi^k}(s),\ \forall s$
   - (Greedy policy improvement) Compute new policy

$$\pi^{k+1}(s) := \arg\max_a [R(s,a) + \gamma \sum_{s'} P(s,a,s') V^{\pi^k}(s')],\ \forall s$$

3. Output optimal policy and value

Convergence of policy iteration using values:

$$\|V^{\pi^k} - V^*\|_\infty \leq \gamma^k \|V^{\pi^0} - V^*\|_\infty$$

Outline

Introduction
- MDP
- Solve an MDP when MDP is known
  - L-operator
  - Value based Iteration
  - Q-value based iteration
  - Discussion

RL: Solve an MDP with samples when MDP is unknown
- Reinforcement Learning
- Q-value based algorithm(and argmaxed-Q policy)
  - Q-learning
  - Discussion of Q-learning
  - Policy iteration with TD-learning
- Policy based algorithm
  - Vanilla Policy gradient methods
  - Conservative Greedy policy improvement algorithm
  - Trust Region Policy Optimization
  - Proximal Policy Optimization

Q-value and Bellman equations

Q-values qualifies the first action under a certain policy for following steps. The introduction of Q-value make it possible for us to get the optimal policy by optimal value without knowing transition model P.

- before: $\pi(s) = \arg\max_a [R(s,a) + \gamma \sum_{s'} P(s,a,s')V(s')]$
- now: $\pi(s) = \arg\max_a Q(s,a)$

Q^*: the expected utility on taking action a in state s, and thereafter acting optimally

$$\begin{aligned} Q^*(s,a) &:= R(s,a) + \gamma \sum_{s' \in S} P(s,a,s')V^*(s') \\ &= R(s,a) + \gamma \sum_{s' \in S} P(s,a,s') \left(\max_{a'} Q^*(s',a') \right) \end{aligned}$$

Q^π: the expected utility on taking action a in state s, and thereafter playing policy π

$$\begin{aligned} Q^\pi(s,a) &:= R(s,a) + \gamma \sum_{s' \in S} P(s,a,s')V^\pi(s') \\ &= R(s,a) + \gamma \sum_{s' \in S} P(s,a,s') \mathbb{E}_{a' \sim \pi(s)}(Q^\pi(s',a')) \end{aligned}$$

Q-value iteration

Iteratively improve estimate of Q-values.

1. Initialize $Q^0 \in \mathbb{R}^{S \times A}$
2. Repeat for $k = 1, 2, \ldots$ until $\|Q^k - Q^{k-1}\|_\infty$ is small, improve the Q-value as

$$Q^k(s,a) = R(s,a) + \gamma \sum_{s'} P(s,a,s') \left(\max_{a'} Q^{k-1}(s',a') \right) \quad \forall s, a$$

3. Output optimal policy by argmax Q values

$$\pi_k(s) = \arg\max_a Q^k(s,a)$$

Policy iteration using Q-values

Iteratively improve estimate of optimal policy.

1. Initialize policy π^0.
2. Repeat for k=1,2,... until $\pi^{k+1} = \pi^k$ (or not much change in policy)

- (Policy evaluation) Compute $Q^{\pi^k}(s, a),\ \forall s, a$
- (Greedy Policy improvement) Compute new policy

$$\pi^{k+1}(s) := \arg\max_a Q^{\pi^k}(s, a), \forall s$$

3. Output optimal policy and value

Outline

Introduction
- MDP
- Solve an MDP when MDP is known
  - L-operator
  - Value based Iteration
  - Q-value based iteration
  - Discussion

RL: Solve an MDP with samples when MDP is unknown
- Reinforcement Learning
- Q-value based algorithm(and argmaxed-Q policy)
  - Q-learning
  - Discussion of Q-learning
  - Policy iteration with TD-learning
- Policy based algorithm
  - Vanilla Policy gradient methods
  - Conservative Greedy policy improvement algorithm
  - Trust Region Policy Optimization
  - Proximal Policy Optimization

Value iteration v.s. Policy iteration

- Main difference
- Which is faster?
  - PI can actually be faster if estimating the performance of a fixed policy is much easier than finding an optimal policy
- Pros and Cons
  - PI can warm start if there is a known good initial policy to start from and improve upon

?

Value based algorithm and Q-value based algorithm

What comes with the introduction of Q-values?

Q-values qualifies the first action under a certain policy for following steps. The introduction of Q-value make it possible for us to get the optimal policy by optimal value without knowing transition model P.

- before: $\pi(s) = \arg\max_a [R(s,a) + \gamma \sum_{s'} P(s,a,s')V(s')]$
- now: $\pi(s) = \arg\max_a Q(s,a)$

Outline

Introduction
- MDP
- Solve an MDP when MDP is known
  - L-operator
  - Value based Iteration
  - Q-value based iteration
  - Discussion

RL: Solve an MDP with samples when MDP is unknown
- Reinforcement Learning
- Q-value based algorithm(and argmaxed-Q policy)
  - Q-learning
  - Discussion of Q-learning
  - Policy iteration with TD-learning
- Policy based algorithm
  - Vanilla Policy gradient methods
  - Conservative Greedy policy improvement algorithm
  - Trust Region Policy Optimization
  - Proximal Policy Optimization

Reinforcement Learning

- Learning from observations

  When we do not know the (R, P) of the MDP of interest, we can interact with the environment to estimate them.

$$R(s, a) = \mathbb{E}(r_t \mid S_t = s, a_t = a)$$

$$P(s, a, s') = P(S_{t+1} = s' \mid S_t = s, a_t = a)$$

- Question
  - How to solve the best policy(implement Q-value iteration and policy iteration) using samples?
  - How to interact with the environment to generate useful samples? (smartly interact)
  - For large scale MDP, use parametric version of Q-values and policy
  - ...

Outline

Introduction
- MDP
- Solve an MDP when MDP is known
  - L-operator
  - Value based Iteration
  - Q-value based iteration
  - Discussion

RL: Solve an MDP with samples when MDP is unknown
- Reinforcement Learning
- Q-value based algorithm(and argmaxed-Q policy)
  - Q-learning
  - Discussion of Q-learning
  - Policy iteration with TD-learning
- Policy based algorithm
  - Vanilla Policy gradient methods
  - Conservative Greedy policy improvement algorithm
  - Trust Region Policy Optimization
  - Proximal Policy Optimization

Intro: Q-value based algorithm

Q-learning and TD-learning:

- sample version of 'Q-value iteration' and 'Policy iteration using Q-values' (both Q-value-based method)
- form of Q-values
  - tabular form when MDP is small
  - parametric function form when MDP is large scale
- optimal policy: argmaxed by final Q-values (a deterministic policy)

Outline

Introduction
- MDP
- Solve an MDP when MDP is known
  - L-operator
  - Value based Iteration
  - Q-value based iteration
  - Discussion

RL: Solve an MDP with samples when MDP is unknown
- Reinforcement Learning
- Q-value based algorithm(and argmaxed-Q policy)
  - Q-learning
  - Discussion of Q-learning
  - Policy iteration with TD-learning
- Policy based algorithm
  - Vanilla Policy gradient methods
  - Conservative Greedy policy improvement algorithm
  - Trust Region Policy Optimization
  - Proximal Policy Optimization

Tabular Q-learning

Q-learning is a **sample-based version of Q-value iteration**.

- recall the update step in Q-learning

$$Q^k(s,a) = R(s,a) + \gamma \sum_{s'} P(s,a,s') \left(\max_{a'} Q^{k-1}(s',a') \right) \quad \forall s, a$$

- A naive method
  For $k = 1, 2, ...$, observe s_k, take a random action a_k(uniformly choose from the action space), observe reward r_k and next state s_{k+1}, **improve the one-point Q-value at** (s_k, a_k) as

$$Q^k(s_k,a_k) = r_t + \gamma \left(\max_{a'} Q^{k-1}(s_{k+1},a') \right)$$

  - unbiased but large variance

Tabular Q-learning

Two ways to make the algorithm more steady and efficient

- Slow down the update(e.g. with learning rate α_k)[6]

$$Q^k(s_k, a_k) = (1-\alpha_k)Q^{k-1}(s_k, a_k)+\alpha_k \left(r_k + \gamma(\max_{a'} Q^{k-1}(s_{k+1}, a'))\right)$$

- Choose action a_k smartly(e.g. ϵ-greedy)

$$a_t = \arg\max_a Q^{t-1}(s_t, a), \text{ w.p. } 1-\epsilon$$

[6]The update can also be expressed in a Temporal difference(TD) sense

$$Q^k(s_k, a_k) \leftarrow Q^{k-1}(s_k, a_k) + \alpha_k \delta$$

where TD is $\delta = r_k + \gamma(\max_{a'} Q^{k-1}(s_{k+1}, a')) - Q^{k-1}(s_k, a_k)$

Tabular Q-learning Algorithm

Iteratively improve estimate of Q-values.

1. Initialize $Q^0 \in \mathbb{R}^{S \times A}$
2. Repeat $k = 1, 2, \ldots$ until $\|Q^k - Q^{k-1}\|_\infty$ is small

- observe s_k, take action a_k using ϵ-greedy policy

$$a_k = \arg\max_a Q^{k-1}(s_k, a), \text{ w.p. } 1 - \epsilon$$

- observe reward r_k and next state s_{k+1}
- improve the one-point Q-value at (s_k, a_k) as

$$Q^k(s_k, a_k) \leftarrow (1-\alpha_k)Q^{k-1}(s_k, a_k) + \alpha_k \left(r_k + \gamma(\max_{a'} Q^{k-1}(s_{k+1}, a')) \right)$$

3. Output policy π_k, where

$$\pi_k(s) = \arg\max_a Q^k(s, a)$$

Large-scale Q-learning

If we are facing large state space, the Q-tabular would become huge and hard to update. How to reduce the dimension? By **parameterizing tabular Q-values into a model**

$$Q_\theta(s,a) = f_\theta(x(s,a))$$

where the vector $x(s,a)$(feature embedding) can have less dimension, and $f_\theta(\cdot)$ can be any function of the vector(e.g. Linear regression or DNN). So **updating Q tabular** $\rightarrow$ **updating** θ **by gradient of loss**.

$$Q^k(s_k,a_k) \leftarrow (1-\alpha_k)Q^{k-1}(s_k,a_k) + \alpha_k\left(r_t + \gamma(\max_{a'} Q^{k-1}(s_{k+1},a')\right)$$

$$L(\theta_{k-1}) = L(Q_{\theta_{k-1}}(s_k,a_k), r_k + \gamma(\max_{a'} Q_{\theta_{k-1}}(s_{k+1},a'))$$

So the parameter is updated as

$$\theta_k \leftarrow \theta_{k-1} - \alpha_k \nabla_\theta L(\theta_{k-1})$$

Large-scale Q-learning

When loss is defined by $L(x, y) = \frac{1}{2}(x - y)^2$, the parameter is updated as

$$\theta_k \leftarrow \theta_{k-1} + \alpha_k \delta_k \nabla_\theta Q_{\theta_{k-1}}(s, a)$$

where

- TD target: $y = r_k + \gamma \max_{a'} Q_{\theta_{k-1}}(s_{k+1}, a')$
- TD error: $\delta_k = r_k + \gamma \max_{a'} Q_{\theta_{k-1}}(s_{k+1}, a') - Q_{\theta_{k-1}}(s_k, a_k)$
- learning rate/step size: α_k

Large-scale Q-learning Algorithm

Deep Q-learning algorithm (DQN)

Iteratively improve estimate of parameter θ.

1. Initialize θ^0
2. Repeat for $k = 1, 2, \ldots$ until $\|\theta^k - \theta^{k-1}\|_\infty$(or its gradient, etc) is small

- observe s_k, take action a_k using ϵ-greedy policy

$$a_k = \arg\max_a Q^{\theta_{k-1}}(s_t, a), \text{ w.p. } 1 - \epsilon$$

- observe reward r_k and next state s_{k+1}
- improve the parameter as

$$\theta_k \leftarrow \theta_{k-1} - \alpha_k \nabla_\theta L(\theta_{k-1})$$

3. Output optimal policy π_k, where

$$\pi_k(s) = \arg\max_a Q^{\theta_k}(s, a)$$

Large-scale Q-learning Algorithm

implementation tips

Outline

Introduction
- MDP
- Solve an MDP when MDP is known
  - L-operator
  - Value based Iteration
  - Q-value based iteration
  - Discussion

RL: Solve an MDP with samples when MDP is unknown
- Reinforcement Learning
- Q-value based algorithm(and argmaxed-Q policy)
  - Q-learning
  - Discussion of Q-learning
  - Policy iteration with TD-learning
- Policy based algorithm
  - Vanilla Policy gradient methods
  - Conservative Greedy policy improvement algorithm
  - Trust Region Policy Optimization
  - Proximal Policy Optimization

Over-estimation bias in Q-learning and Double Q-learning

A fact: **Suppose that we have unbiased estimate of the optimal Q-values, after one Q-learning update, the new Q-value will be positively biased.**

proof:

$$\hat{Q}^{new}(s,a) \leftarrow (1-\alpha)\hat{Q}(s,a) + \alpha\left(r + \gamma(\max_{a'} \hat{Q}(s',a'))\right)$$

$$\begin{aligned} E[\hat{Q}^{new}(s,a)] &= (1-\alpha)\mathbb{E}[\hat{Q}(s,a)] + \alpha\mathbb{E}\Big[(r + \gamma(\max_{a'} \hat{Q}(s',a'))\Big] \\ &= (1-\alpha)Q^*(s,a) + \alpha R(s,a) + \alpha\gamma\mathbb{E}[\max_{a'} \hat{Q}(s',a')] \end{aligned}$$

suppose with positive probability some action satisfies

$$Q^*(s',a) + \epsilon_a > Q^*(s',a^*) + \epsilon_{a^*}, \;\; Q^*(s',a^*) = \max_{a'} Q^*(s',a')$$

then

$$\mathbb{E}[\max_{a'} \hat{Q}(s',a')] = \mathbb{E}[\max_{a'}(Q^*(s',a') + \epsilon_{s',a'})] > \max_{a'} Q^*(s',a')$$

$$\mathbb{E}[\hat{Q}^{new}(s,a)] > Q^*(s,a)$$

Over-estimation bias in Q-learning and Double Q-learning

The over-estimation is due to the algorithm that take the expectation of the max on estimated Q-values. Double Q-learning can mitigate this issue by decoupling the action selection from the evaluation.

Double Q-learning: Let $\hat{Q}_1, \hat{Q}_2$ be independent unbiased estimates of the optimal Q-values, update as

$$\hat{Q}_1^{new}(s,a) \leftarrow (1-\alpha)\hat{Q}_1(s,a) + \alpha\left(r + \gamma \hat{Q}_2(s', \arg\max_{a'} \hat{Q}_1(s',a'))\right)$$

can show that

$$\mathbb{E}[\hat{Q}_2(s', \arg\max_{a'} \hat{Q}_1(s',a')) \leq \max_{a'} Q^*(s',a')$$

$$\mathbb{E}[\hat{Q}_1^{new}(s,a)] \leq Q^*(s,a)$$

Double Q-learning has under-estimation bias, which is better than over-estimation(why?).

Tabular v.s. Functional Form

The form of Q-values can be tabular or some functional form based on the scale of MDP.

- Tabular setting
  -
  - hard to handle large scale settings
- Function approximation
  - ability of Generalization

projected bellman equation

Outline

Introduction
- MDP
- Solve an MDP when MDP is known
  - L-operator
  - Value based Iteration
  - Q-value based iteration
  - Discussion

RL: Solve an MDP with samples when MDP is unknown
- Reinforcement Learning
- Q-value based algorithm(and argmaxed-Q policy)
  - Q-learning
  - Discussion of Q-learning
  - Policy iteration with TD-learning
- Policy based algorithm
  - Vanilla Policy gradient methods
  - Conservative Greedy policy improvement algorithm
  - Trust Region Policy Optimization
  - Proximal Policy Optimization

Tabular TD-learning

Recall the policy iteration with Q-values, there is a step called policy evaluation, where we **compute the Q-values for each state and action for a given policy** $Q^\pi(s, a),\ \forall s, a$.

TD-learning is a **sample-based approximate version of policy evaluation**, where we **estimate the Q-values for each state and action for a given policy**

Computing Q-values of a fixed policy π:

- Monte Carlo method
- Q-value-iteration: Based on Bellman equation (DP)

$$Q^\pi(s, a) = R(s, a) + \gamma \sum_{s' \in S} P(s, a, s') \mathbb{E}_{a' \sim \pi(s)}(Q^\pi(s', a'))$$

Tabular TD-learning algorithm for learning Q-values of policy π

A subroutine of policy iteration using Q-values

[?]

Iteratively improve estimate of Q-values.

1. Initialize $Q^0 \in \mathbb{R}^{S \times A}$
2. In every iteration $k = 1, 2, \ldots$

- observe s_k, take action a_k using ϵ-greedy policy

$$a_k = \arg\max_a Q^{k-1}(s_k, a), \text{ w.p. } 1 - \epsilon$$

- observe reward r_k and next state s_{k+1}
- improve the one-point Q-value at (s_k, a_k) as

$$Q^k(s_k, a_k) \leftarrow (1-\alpha_k)Q^{k-1}(s_k, a_k) + \alpha_k \left(r_k + \gamma(\mathbb{E}_{a' \sim \pi(s_{k+1})} Q^{k-1}(s_{k+1}, a'))\right)$$

3. Stop if $\|Q^k - Q^{k-1}\|_\infty$ is small

Large-scale TD-learning

A subroutine of policy iteration using Q-values

Outline

Introduction
- MDP
- Solve an MDP when MDP is known
  - L-operator
  - Value based Iteration
  - Q-value based iteration
  - Discussion

RL: Solve an MDP with samples when MDP is unknown
- Reinforcement Learning
- Q-value based algorithm(and argmaxed-Q policy)
  - Q-learning
  - Discussion of Q-learning
  - Policy iteration with TD-learning
- Policy based algorithm
  - Vanilla Policy gradient methods
  - Conservative Greedy policy improvement algorithm
  - Trust Region Policy Optimization
  - Proximal Policy Optimization

Intro: Policy based algorithm

(Sutton et al. 1999)

Q-learning and TD-learning:

- Q-value-based method, solving Bellman equations, not targeting policy directly
- deterministic policy (argmaxed by Q-values)
- Limitation: an arbitrarily small change in the estimated value can cause the action to be or not to be selected, hard to establish convergence assurances

Policy based algorithms:

- can be **stochastic policy**, and can handle continuous action
- can use **function form of policy** and **targeting value directly to find optimal policy**

Functional Form of Policy

Parameterizing the policy space as

$$\pi_\theta : S \times A \to R, \ \sum_a \pi_\theta(s, a) = 1$$

- Examples
  - Discrete action space $\pi_\theta(s, a) = \frac{e^{f_{\theta a}(x_s)}}{\sum_{a' \in A} e^{f_{\theta a'}(x_s)}}$
  - Continuous unrestricted action space $\pi_\theta(s) = \mathcal{N}(f_\theta(s), \sigma^2)$

Therefore, goal of finding optimal policy is equivalent to finding the optimal parameter that,

$$\theta^* = \arg\max_\theta V^{\pi_\theta}(s)$$

A natural question: Can we compute/estimate the gradient?

$$\theta_{k+1} \leftarrow \theta_k + \alpha_k \nabla_\theta V^{\pi_\theta}(s)$$

Outline

Introduction
- MDP
- Solve an MDP when MDP is known
  - L-operator
  - Value based Iteration
  - Q-value based iteration
  - Discussion

RL: Solve an MDP with samples when MDP is unknown
- Reinforcement Learning
- Q-value based algorithm(and argmaxed-Q policy)
  - Q-learning
  - Discussion of Q-learning
  - Policy iteration with TD-learning
- Policy based algorithm
  - Vanilla Policy gradient methods
  - Conservative Greedy policy improvement algorithm
  - Trust Region Policy Optimization
  - Proximal Policy Optimization

Policy gradient methods - Finite horizon

Goal:

$$V^{\pi}(s_1) = \mathbb{E}\left(\sum_{t=1}^{H} \gamma^{t-1} r_t \mid s_1\right)$$

Theorem: for a policy π_θ

$$\nabla_\theta V^{\pi_\theta}(s_1) = \mathbb{E}_\tau \left[R(\tau) \sum_{t=1}^{H} \nabla_\theta \log(\pi_\theta(s_t, a_t)) \mid s_1 \right]$$

where $R(\tau)$ can be estimated, $\nabla_\theta \log(\pi_\theta(s, a))$ can be computed.

Policy gradient methods - Finite horizon

Vanilla Policy gradient algorithm, a.k.a. REINFORCE (Sutton et al. 1999)

1. Initialize θ^0
2. In every iteration $k = 1, 2, \ldots$

- Execute current policy π_θ to obtain several sample trajectories $\tau_i, i = 1, ..., m$
- observe rewards and calculate the empirical total discounted reward $\hat{R}(\tau_i)$ of each trajectory $\hat{R}(\tau) = r_1 + \gamma r_2 + ...\gamma^{H+1} r_H$
- Use these sample trajectories and chosen baseline to compute an unbiased gradient estimator using Policy gradient theorem

$$\hat{\mathbf{g}}_k = \frac{1}{m} \sum_{i=1}^{m} \hat{R}(\tau_i) \sum_{t=1}^{H} \nabla_\theta \log\big(\pi_\theta(s_t^i, a_t^i)\big)$$

- Update:

$$\theta_{k+1} \leftarrow \theta_k + \alpha_k \hat{\mathbf{g}}_k.$$

Policy gradient methods - Infinite horizon

Goal:

$$V^{\pi_\theta}(s_1) = \lim_{T\to\infty} \mathbb{E}\left(\sum_{t=1}^{T} \gamma^{t-1} r_t \mid s_1\right)$$
$$= \sum_s d^{\pi_\theta}(s \mid s_1) \sum_a \pi_\theta(s,a) R(s,a)$$

where $d^{\pi_\theta}(s \mid s_1)$ is the total discounted prob of being in s under π

$$d^{\pi_\theta}(s \mid s_1) = \lim_{T\to\infty} \sum_{t=1}^{T} \gamma^{t-1} P(s_t = s \mid s_1, \pi)$$

Theorem: for a given policy π_θ, the gradient of value can be calculated as

$$\nabla_\theta V^{\pi_\theta}(s_1) = \sum_s d^{\pi_\theta}(s \mid s_1) \sum_a Q^{\pi_\theta}(s,a) \nabla_\theta \pi_\theta(s,a)$$
$$= \sum_s d^{\pi_\theta}(s \mid s_1) \sum_a \pi_\theta(s,a) \left[Q^{\pi_\theta}(s,a) \nabla_\theta \log(\pi_\theta(s,a))\right]$$
$$= \frac{1}{1-\gamma} \mathbb{E}_{s\sim(1-\gamma)d^{\pi_\theta}, a\sim\pi_\theta(s)} \left[Q^{\pi_\theta}(s,a) \nabla_\theta \log(\pi_\theta(s,a))\right]$$

where $Q^{\pi_\theta}(s,a)$ can be estimated, $\nabla_\theta \log(\pi_\theta(s,a))$ can be computed.

Policy gradient methods - Infinite horizon

Vanilla Policy Gradient Algorithm

1. Initialize policy parameter θ^0.
2. In each iteration $k = 1, 2, \ldots$:

- Execute current policy π_θ to obtain several sample trajectories $\tau_i, i = 1, ..., m$ (on-policy method)
- For any given sample trajectory i, use observed rewards to compute an unbiased estimate of $Q(s_t, a_t)$

$$\hat{Q}_t^i := \sum_{t'=t}^{T-1} \gamma^{t'-t} r_{t'}^i.$$

- Use $\hat{Q}_t^i$ to compute a gradient estimator $\hat{g}_k$

$$\hat{\mathbf{g}}_k = \frac{1}{m} \sum_{i=1}^{m} \sum_{t=1}^{T} \gamma^{t-1} \hat{Q}_t^i \nabla_\theta \log\left(\pi_\theta(s_t^i, a_t^i)\right)$$

Update:

$$\theta_{k+1} \leftarrow \theta_k + \alpha_k \hat{\mathbf{g}}_k.$$

Estimation error in Policy gradient methods

Policy gradient methods relies on approximate Q-values/advantages and can be misled by estimation errors.
An example:

Outline

Introduction
- MDP
- Solve an MDP when MDP is known
  - L-operator
  - Value based Iteration
  - Q-value based iteration
  - Discussion

RL: Solve an MDP with samples when MDP is unknown
- Reinforcement Learning
- Q-value based algorithm(and argmaxed-Q policy)
  - Q-learning
  - Discussion of Q-learning
  - Policy iteration with TD-learning
- Policy based algorithm
  - Vanilla Policy gradient methods
  - Conservative Greedy policy improvement algorithm
  - Trust Region Policy Optimization
  - Proximal Policy Optimization

Intro: Conservative Policy Iteration

- Problems of policy gradient methods
  - update policy by gradient $\neq$ improvement of policy monotonicly, only a local ascent direction
  - also, in practice, the existence of estimation errors can misled the update direction
- Can we design an algorithm that
  - guaranteed to improve some performance measure at every step?
  - and can control the performance changes and prevent large degradations? under the existence of estimation error

Advantage function and Performance difference lemma

- Advantage function

  The advantage of action a in state s is defined as

$$A^{\pi}(s,a) = Q^{\pi}(s,a) - V^{\pi}(s)$$

- Performance difference lemma

  For any policy $\tilde{\pi}, \pi$ start at same state s_1

$$\begin{aligned} V^{\tilde{\pi}}(s_1) - V^{\pi}(s_1) &= \sum_s d^{\tilde{\pi}}(s) \sum_a \tilde{\pi}(s,a) A^{\pi}(s,a) \\ &= \frac{1}{(1-\gamma)} \mathbb{E}_{s \sim (1-\gamma) d^{\tilde{\pi}}} \sum_a \tilde{\pi}(s,a) A^{\pi}(s,a) \end{aligned}$$

  which shows that the difference of two policies can be expressed by the expected advantage of one policy on the generated trajectory distribution of another policy[7].

[7] But think of the question if we wish to estimate the improve of a new unknown policy $\tilde{\pi}$ with current policy π, we can only get samples from $s \sim d^{\pi}, A^{\pi}$ instead of $s \sim d^{\tilde{\pi}}, A^{\pi}$, so we need further theorem

Policy improvement theorem

(Kakade and Langford 2002)

This theorem **quantifies the least change of value of the new mixture policy with the original policy** with step size α and another policy π'.

Given any two policies π,π', let π^{new} be

$$\pi_\alpha^{\text{new}} = (1-\alpha)\pi + \alpha\pi'$$

and for $\alpha \in [0,1]$ we have

$$V^{\pi_\alpha^{\text{new}}}(s_1) - V^{\pi}(s_1) \geq \frac{\alpha}{1-\gamma} A_\pi(\pi') - \frac{\alpha^2}{(1-\gamma)^2} 2\gamma A_\pi^{\max}$$

where

$$A_\pi(\pi') = \mathbb{E}_{s\sim(1-\gamma)d^\pi}\left[\sum_a \pi'(s,a) A^\pi(s,a)\right]$$

$$A_\pi^{\max} = \max_s \left|\sum_a \pi'(s,a) A^\pi(s,a)\right|$$

Conservative Greedy policy improvement algorithm

(Kakade and Langford 2002)

Policy improvement theorem helps us design an algorithm that always ensure a positive improvement in each policy update step by choosing π' and α, where π' is the 'greedy' part and α make it 'properly conservative'.

- For π', intuitively we can choose a policy that is greedy

$$\pi'(s) = \arg\max_a A^\pi(s, a)$$

  which guaranteed that $A_\pi(\pi')$ is maximized.

- Given π' is a greedy policy, for α, let R be an upper bound on rewards, so that $A_\pi(\pi') \leq A_\pi^{\max} \leq \frac{R}{1-\gamma}$. Then, set

$$\alpha = \frac{A_\pi(\pi')(1-\gamma)^2}{4R}$$

so that

$$V(\pi^{new}) - V(\pi) \geq \frac{\alpha}{(1-\gamma)} A_\pi(\pi') - \frac{2\alpha^2 A_\pi^{max}}{(1-\gamma)^2} \geq \frac{A_\pi(\pi')^2(1-\gamma)}{8R} \geq 0$$

where $A_\pi(\pi')$ can be estimated by simulation/sampling from current policy.

Conservative Greedy policy improvement algorithm

(Kakade and Langford 2002)

1. Initialize policy π^0
2. In every iteration k = 0,1,2,... (k is omitted for simplicity)

- (policy evaluation) Play policy π from starting state distribution μ to generate sample trajectories. Estimate advantage function $\hat{A}^{\pi}(s,a)$(by Monte Carlo or TD-learning)
- (policy improvement) Compute π' as the argmax policy

$$\max_{\pi' \in \Pi} \hat{\mathbb{E}}_{s \sim (1-\gamma) d^{\pi,\mu}} \left[\sum_a \pi'(s,a) \hat{A}^{\pi}(s,a) \right] =: \hat{A}$$

and step size as

$$\alpha = \hat{A} \frac{(1-\gamma)^2}{4R}$$

Update policy:

$$\pi \leftarrow (1-\alpha)\pi + \alpha\pi'$$

3. stop when $\hat{A}$ is very small

From single start state to general

- Before, we discussed the best policy regard to a single start state $J_{s_1}(\pi) = V^\pi(s_1)$, and $d^\pi_{s_1}(s) = \sum_{t=1}^{\infty} \gamma^{t-1} P(s_t = s \mid s_1, \pi)$
- Extend to a general setting, where the starting state is from a distribution μ

$$\eta_\mu(\pi) = \mathbb{E}_{s_1 \sim \mu}[V^\pi(s_1)] = \sum_{s_1} \mu(s_1) V^\pi(s_1)$$

where

$$d^\pi_\mu(s) = \sum_{s_1} \mu(s_1) d^\pi_{s_1}(s)$$

- general Performance difference lemma

$$\eta_\mu(\tilde{\pi}) - \eta_\mu(\pi) = \frac{1}{1-\gamma} \mathbb{E}_{s \sim d_{\tilde{\pi},\mu}, a \sim \tilde{\pi}(s)} \left[A^\pi(s,a) \right]$$

- general Policy improvement theorem for $\alpha \leq 1$

$$\eta_\mu(\pi_{new}) - \eta_\mu(\pi) \geq \frac{\alpha}{1-\gamma} \left(\mathbb{A}_{\pi,\mu}(\pi') - \frac{2\alpha\gamma\varepsilon}{1-\gamma(1-\alpha)} \right)$$

where

$$\mathbb{A}_{\pi,\mu}(\pi') \equiv \mathbb{E}_{s \sim d_{\pi,\mu}, a \sim \pi'(s)} \left[A^\pi(s,a) \right]$$

Local improvement vs. optimality

Q: policy improvement and global monotone?

Parameterized Conservative Greedy policy improvement algorithm

$$\theta^*_{k+1} = \arg\max_{\theta} \mathbb{E}_{s\sim(1-\gamma)d^{\pi_{\theta_k}}} \left[\sum_a \pi_\theta(s,a)\, A^{\pi_{\theta_k}}(s,a) \right]$$

Intro: Smart-conservative-greedy parameterized algorithm

TRPO and PPO

- Conservative greedy policy improvement algorithm
  - every iteration adds a new policy to maintain, costing a lot of space
  - the greedy policy part may be hard to solve
- Trust Region Policy Optimization (TRPO)
  - uses a parametric form of policy, just need to maintain and update the policy parameter
  - can be seemed as a smart-conservative-greedy algorithm by **bounding the difference between old and new policy**
  - theoretically good
- Proximal Policy Optimization (PPO)
  - also uses a parametric form of policy
  - similar object to maximize as TRPO, but use its pessimistic bound, **clip on the goal directly instead of using KL-divergence constraint**
  - compared to TRPO, a little bit ad hoc, but more technically efficient, easy to implement

Outline

Introduction
- MDP
- Solve an MDP when MDP is known
  - L-operator
  - Value based Iteration
  - Q-value based iteration
  - Discussion

RL: Solve an MDP with samples when MDP is unknown
- Reinforcement Learning
- Q-value based algorithm(and argmaxed-Q policy)
  - Q-learning
  - Discussion of Q-learning
  - Policy iteration with TD-learning
- Policy based algorithm
  - Vanilla Policy gradient methods
  - Conservative Greedy policy improvement algorithm
  - Trust Region Policy Optimization
  - Proximal Policy Optimization

Trust Region Policy Optimization (TRPO)

(Schulman, Levine, et al. 2015)

This theorem quantifies quantifies the least change of value for any two policies $\pi, \tilde{\pi}$[8] with policy advantage and policy KL-divergence of the two policies,

$$\eta(\tilde{\pi}) - \eta(\pi) \geq A_{\pi}(\tilde{\pi}) - \frac{2\gamma\varepsilon}{(1-\gamma)^2} D_{KL}^{\max}(\tilde{\pi}, \pi)$$

So we can design an Approximate Policy Iteration algorithm that guarantees increasing value:

In each iteration k = 0,1,2,... Solve the constrained optimization problem

$$\pi_{k+1} = \arg\max_{\pi} \left[A_{\pi_k}(\pi) - \frac{2\gamma\varepsilon}{(1-\gamma)^2} D_{KL}^{\max}(\pi_k, \pi) \right]$$

[8] can be proved by extending the policy improvement theorem, $\alpha = D_{TV}^{max}(\pi, \pi^{new})$

Trust Region Policy Optimization (TRPO)

(Schulman, Levine, et al. 2015)

Now consider parameterized policies π_θ, and uses a constraint on the KL divergence rather than a penalty[9].

In each iteration $k = 0, 1, 2...$

$$\theta_{k+1} = \begin{array}{ll} \arg\max_\theta & \mathbb{E}_{s\sim\pi_{\theta_k}}\left[\sum_a \pi_\theta(s,a) A^{\pi_{\theta_k}}(s,a)\right] \\ \text{s.t.} & D_{\text{KL}}(\pi_\theta(\cdot|s), \pi_{\theta_k}(\cdot|s)) \le \delta \end{array}$$

In practice, use the sample-based estimation of $\mathbb{E}_{s\sim\pi_{\theta_k}}$ and $A^{\pi_{\theta_k}}$ and the sample-based estimation of average KL-divergence.

$$\theta_{k+1} = \begin{array}{ll} \arg\max_\theta & \hat{\mathbb{E}}_{s\sim\pi_{\theta_k}}\left[\sum_a \pi_\theta(s,a) \hat{A}^{\pi_{\theta_k}}(s,a)\right] \\ \text{s.t.} & \hat{\mathbb{E}}_{s\sim\pi_{\theta_k}}\left[D_{\text{KL}}(\pi_\theta(\cdot|s), \pi_{\theta_k}(\cdot|s))\right] \le \delta \end{array}$$

[9] In practice, if using the penalty coefficient $\frac{2\gamma\varepsilon}{(1-\gamma)^2}$, the step sizes would be very small

Outline

Introduction
- MDP
- Solve an MDP when MDP is known
  - L-operator
  - Value based Iteration
  - Q-value based iteration
  - Discussion

RL: Solve an MDP with samples when MDP is unknown
- Reinforcement Learning
- Q-value based algorithm(and argmaxed-Q policy)
  - Q-learning
  - Discussion of Q-learning
  - Policy iteration with TD-learning
- Policy based algorithm
  - Vanilla Policy gradient methods
  - Conservative Greedy policy improvement algorithm
  - Trust Region Policy Optimization
  - Proximal Policy Optimization

Proximal Policy Optimization (PPO)

(Schulman, Wolski, et al. 2017)

Let $r_k(\theta) := \frac{\pi_\theta(s,a)}{\pi_{\theta_k}(s,a)}$, recall that TRPO maximize (under constraint)

$$\theta_{k+1} = \arg\max_\theta \hat{\mathbb{E}}_{s,a\sim\pi_{\theta_k}} \left[\frac{\pi_\theta(s,a)}{\pi_{\theta_k}(s,a)} \hat{A}^{\pi_{\theta_k}}(s,a) \right],$$

$$L^{CPI}(\theta) = \hat{\mathbb{E}}_k \left[r_k(\theta)\hat{A}_k \right]$$

PPO instead solve the unconstrained problem by maximize

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_k \left[\min\left(r_k(\theta)\hat{A}_k, \text{clip}(r_k(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_k \right) \right]$$

What does clip-then-min mean?

PPO (Schulman, Wolski, et al. 2017)

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_k \left[\min \left(r_k(\theta)\hat{A}_k, \text{clip}(r_k(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_k \right) \right]$$

- a pessimistic bound in the sense that
  - clip(ignore) when the policy improve
    - if $\hat{A} > 0$ and $r_k > 1+\epsilon$, the final is $(1+\epsilon)\hat{A}$
    - if $\hat{A} < 0$ and $r_k < 1-\epsilon$, the final is $(1-\epsilon)\hat{A}$
  - unclip when policy is worse
    - if $\hat{A} > 0$ and $r_k < 1-\epsilon$, the final is $r_t\hat{A}$
    - if $\hat{A} < 0$ and $r_k > 1+\epsilon$, the final is $r_k\hat{A}$

A soft way of conservative-greedy. By maximizing a pessimistic lower bound on the surrogate objective, PPO avoids the large jump to an estimated-better but actually-worse policy.

Proximal Policy Optimization (PPO)

(Schulman, Wolski, et al. 2017)

alternate between sampling data from the policy and performing several epochs of optimization on the sampled data

1. Initialize policy π^0
2. Repeat for k=1,2,... until convergence

- (policy evaluation) Play policy π from starting state distribution μ to generate sample trajectories. Estimate advantage function $\hat{A}^{\pi}(s, a)$(by Monte Carlo or TD-learning)
- (policy improvement) Optimize surrogate $L(\theta)$, with K epochs and minibatch size $M \leq NT$

$$\theta_{k+1} \leftarrow \theta$$

Actor-critic methods

- Critic-only methods (e.g., Q-learning, TD-learning) use TD-updates with function approximation to estimate Q^* or Q^π. Maintain Deep Q-network.
- Actor-only methods (policy gradient) work with a parameterized policy. Parameters are updated by the estimated gradient of the performance. Maintain Policy Network.

what comes with critic? the old problems of Q-learning

More

For more detailed notes on modern RL methods, see

References

Kakade, Sham and John Langford (2002). "Approximately optimal approximate reinforcement learning". In: *Proceedings of the nineteenth international conference on machine learning*, pp. 267–274.

Schulman, John, Sergey Levine, et al. (2015). "Trust region policy optimization". In: *International conference on machine learning*. PMLR, pp. 1889–1897.

Schulman, John, Filip Wolski, et al. (2017). "Proximal policy optimization algorithms". In: *arXiv preprint arXiv:1707.06347*.

Sutton, Richard S et al. (1999). "Policy gradient methods for reinforcement learning with function approximation". In: *Advances in neural information processing systems* 12.